

lexsync: A cross-platform pipeline for multidimensional lexical optimisation and hardware-timed experiment generation

Pablo Bernabeu

2026-07-23

Psycholinguistic stimulus preparation is fragmented across tools that match words, tools that present them and the separate R and Python ecosystems in which those tools live. We present ‘lexsync’, a toolkit that unifies the whole path from corpus to laboratory, released as a ‘CRAN’-ready R package and a structurally identical ‘PyPI’-ready Python package. From a word-frequency corpus in any of dozens of languages ‘lexsync’ selects stimuli that are matched in parallel across several lexical dimensions (length, frequency, orthographic neighbourhood density and OLD20), counterbalances them, reports the realised match quality and then generates ready-to-run ‘PsychoPy’, ‘OpenSesame’ and browser-based ‘jsPsych’ experiments. The two laboratory targets time-lock their electroencephalography triggers to stimulus onset. The trial is described declaratively as a sequence of events, so the same engine builds factorial word studies, lexical decision with generated pseudowords, priming and self-paced reading from configuration rather than code. The default matching methods, the pseudoword generation and the trial order are deterministic, so the two engines produce byte-identical experiments. We demonstrate this on fifteen worked examples in English, Spanish and Mandarin Chinese (across word, pseudoword, paired and sentence stimuli), one of which shows that the workflow extends to a logographic writing system, while three reproduce published designs from the literature. The generation step requires no presentation software and no hardware, which keeps the whole workflow reproducible. ‘lexsync’ is general-purpose and treats the stimulus set as a reproducible artefact. It is distributed under open licences and includes a machine-readable corpus registry.

1 Introduction

The credibility of psycholinguistic and neurolinguistic research rests on carefully controlled stimuli and on precise timing. Both are laborious to achieve, and the tools that support them are scattered. A researcher who wishes to contrast, say, high- and low-frequency words while equating length and orthographic neighbourhood typically matches the items in one program, lays out the trial structure in another and writes the trigger code that aligns each word with the electroencephalography (EEG) recording by hand. Each hand-off is an opportunity for error and a barrier to reuse. The ‘FAIR’ guiding principles (Wilkinson et al., 2016) (that research objects be findable, accessible, interoperable and reusable) apply as much to this preparatory machinery as to the data it ultimately yields, yet stimulus pipelines are seldom shared in a form that others can run end to end.

This preparatory machinery is where a quiet reproducibility problem lives. A survey of 600 randomly sampled linguistics articles found materials, data and protocols each shared in fewer than 10% of them, with no noticeable rise between 2008/2009 and 2018/2019 (Bochynska et al., 2023). Yet materials are a precondition both for replicating a result and for judging how far it generalises. The gap compounds an analytic one. Quantitative phonetic and psycholinguistic work affords many undocumented analytic choices, from how a measure is quantified to how a model is specified, and exploiting that flexibility inflates false positives (Roettger, 2019) and yields widely divergent conclusions from the same data (Coretta et al., 2023). Transparent, runnable, shareable materials are part of the remedy this literature calls for (Roettger et al., 2019).

There is also a statistical reason to take materials seriously. The items in an experiment are a sample from a larger population of possible items, and a conclusion is meant to hold for that population, not merely for the words actually shown. Treating items as a fixed rather than a random factor inflates Type I error (the language-as-fixed-effect fallacy (Clark, 1973)), and recent work recasts the same point as a *generalizability crisis*: researchers routinely model participants as a random factor but not stimuli, and so claim more than their designs license (Yarkoni, 2022). A tool that makes the construction of a matched, balanced item set explicit, reproducible and shareable is therefore part of the methodological infrastructure that careful inference requires.

Two families of tool address parts of the problem. The first matches or equates word stimuli on lexical properties. ‘LexOPS’ (Taylor et al., 2020) generates controlled word lists for arbitrary factorial designs, with both per-dimension tolerances and weighted multivariate Euclidean matching; ‘LIBRA’ (Lintz et al., 2021) equates lists across conditions using a genetic (evolutionary) algorithm; and ‘LASTU’ (Itkonen et al., 2024) provides searchable access to a language-specific lexicon. These tools are mature and widely used, but they end at the stimulus list. None generates the experiment that presents it, and none is available in both the R and the Python ecosystems. Their language coverage follows the lexical database each ships with rather than the method itself, and both ‘LexOPS’ and ‘LIBRA’ will work from any suitably formatted dataset, in any language for which the user can supply the variables. The second family builds and runs experiments. ‘PsychoPy’ (Peirce et al., 2019) and ‘OpenSesame’ (Mathôt et al., 2012) offer precise spatial and temporal control over stimulus presentation and support the hardware triggers that ERP research requires, but they take the stimulus set as given and do nothing to construct or to balance it.

The gap, then, is one of integration rather than of capability. No single, installable framework spans many-language corpus access, parallel multidimensional matching, counterbalancing and the automated generation of hardware-timed presentation scripts, and none does so across both the R and the Python ecosystems and both major open presentation platforms. A second, subtler gap concerns timing. In one longitudinal EEG study of artificial language learning, for instance (González Alonso et al., 2025), EEG markers were sent from an experiment item that followed the word, so that only the first marker was even approximately aligned with stimulus onset and the others were sequence-ordered. Their timing then followed the durations of the preceding events rather than the flip that put the word on screen, a dependency that binding the trigger to the flip itself removes.

We present ‘lexsync’, a toolkit that fills these gaps. It is released as two independent but structurally identical packages (a ‘CRAN’-ready R package and a ‘PyPI’-ready Python package) that share module names, function names and behaviour. From a corpus ‘lexsync’ selects matched stimuli, counterbalances and documents them. It then writes a ‘PsychoPy’ script in which each onset trigger is bound, using `win.callOnFlip`, to the exact screen flip on which the critical stimulus appears, together with a complete ‘OpenSesame’ experiment. The default

matching methods and the trial ordering are deterministic, so the R and Python engines select identical materials, order them identically and write byte-identical experiment files. The trial itself is described declaratively. A design sets out an ordered sequence of trial *events*, and each presentation backend renders that sequence, so the same machinery produces a factorial word study, a lexical-decision task with generated pseudowords, a primed design or a self-paced reading study from configuration rather than new code. ‘lexsync’ is thus a reusable, FAIR-compliant tool for many psycholinguistic paradigms and languages rather than a single study’s script. The longitudinal study above is one of fifteen worked examples, and one of the three that rebuild a published design.

2 The lexsync packages

2.1 Architecture and cross-language design

Each package is organised into the same modules: **querying** (load a lexicon or an item table and compute lexical dimensions), **matching** (the constraint engine), **generation** (deterministic pseudoword generation), **paradigms** (the trial-event registry), **counterbalancing**, **validation** (match-quality reporting), **scripting** (experiment generation), **datasheet** (the materials datasheet and pre-registration template), **logging** (the automated run log) and **corpora** (the registry). A single orchestrator, **run_pipeline**, drives them for one design, and **run_all** applies it in an explicit loop over every design configuration in a directory, so that adding a language, a design or a paradigm is a matter of supplying a configuration file (and, where relevant, a lexicon or item table) rather than writing code. The two packages are deliberately lean: the R package depends only on ‘readr’, ‘yaml’, ‘stringdist’, ‘stringi’, ‘jsonlite’ and ‘digest’ beyond base R, and the Python package on ‘pandas’, ‘numpy’, ‘PyYAML’, ‘rapidfuzz’ and ‘scipy’.

2.2 A many-language corpus registry

The quality of the matched stimuli depends on the corpus from which they are drawn. ‘lexsync’ reads a machine-readable registry that records, for each corpus it knows about, its language, licence, citation, verification status and the column names that map the source file onto the package schema. Two connectors are provided. The first reads the delimited files of the curated SUBTLEX family and the ‘openlexicon’ collection (for example SUBTLEX-ESP for Spanish (Cuetos et al., 2011) and SUBTLEX-US for American English (Brysbaert & New, 2009)), each individually citable and made freely available by its authors subject to attribution, on terms equivalent to CC BY-SA. No entry in that family currently registers a directly addressable delimited file, so the registry marks the corpora the connector cannot read and gives a landing page instead. SUBTLEX-UK (Heuven et al., 2014), for one, is published only as zipped archives, so the researcher downloads the file once and passes it to `load_lexicon()`. The second connector is the ‘wordfreq’ package (Speer, 2022), which supplies frequencies through a single dependency for the thirty languages the registry currently admits, among them Greek, Russian, Arabic, Hebrew, Hindi, Chinese, Japanese and Korean. The package itself covers around forty, and the rest are enabled by adding their language code to the registry entry. The demonstrations below build their English, Spanish and Mandarin Chinese lexica with ‘wordfreq’, which aggregates subtitle and web corpora and, for some languages including English and Chinese, the corresponding SUBTLEX lists. The registry documents the curated corpora (among them SUBTLEX-CH for Chinese (Cai & Brysbaert, 2010)) as alternatives. Small example lexica are bundled inside each package for offline use and testing, which keeps the installed packages

within distribution size limits. A lexicon for any of the admitted ‘wordfreq’ languages is derived on demand into a user cache, an ingestion step that runs in Python, and the R package reads the result as an ordinary corpus.

2.3 The multidimensional matching algorithm

For every word ‘lexsync’ records its length, its frequency and two orthographic-neighbourhood measures: Coltheart’s N , the number of words of the same length differing by a single letter (Coltheart et al., 1977), and OLD20, the mean Levenshtein distance to the twenty nearest neighbours (Yarkoni et al., 2008). Both are computed transparently from the lexicon rather than taken on trust, and the reference set is the full loaded lexicon rather than the filtered candidate pool the items are drawn from. A word is excluded from its own neighbourhood, and where fewer than twenty other forms are available OLD20 averages over as many as there are. The dimension set is open. It also includes an orthographic syllable estimate (the number of maximal vowel runs, counting y and the accented Latin vowels as vowels) and a bigram-frequency measure, the mean over a word’s adjacent letter bigrams of that bigram’s share of all bigram occurrences in the lexicon, which approximates phonotactic probability from word forms rather than from frequency-weighted tokens. Externally-sourced norms (concreteness, age of acquisition, valence) can be joined on through a norm-table connector so the matcher equates on them too. A sixth corpus demonstration puts two of the added dimensions to use, a frequency contrast matched on length, syllable count and bigram frequency. Both engines select the same items for it.

A design declares its candidate pool by filtering the lexicon on any recorded dimension, and then defines each condition by a range or a set of permitted values on one or more dimensions, so each condition reaches the matcher as its own subpool. The first condition listed is the anchor. Its items are not drawn at random, and they are not taken from one end of the subpool either. The subpool is sorted by the first dimension the condition is defined on, with ties broken by the UTF-8 bytes of the word, and the n items are taken at evenly spaced ranks along it, so the anchor spans the subpool rather than clustering at one end.

Matching then proceeds in two deterministic stages. First, a tolerance pre-filter keeps only candidates that fall within $M \pm k \cdot SD$ of the anchor condition on each control dimension, generalising the windowed selection used by González Alonso et al. (2025). The schema sets k to 1 for frequency and to 2 for length, Coltheart’s N and OLD20, and any dimension it does not name falls back to 2. A design file may override these values, as the reproduction of that study below does. If a condition holds fewer candidates inside its window than the design asks for, the window is dropped for that condition and the match is made over its whole subpool, so a window set too narrow costs matching precision rather than the run. The matcher stops with an error only when the subpool itself is too small, or too incomplete on the matched dimensions, to fill the design, and the remedy is then to widen the pool filters or to ask for fewer items per condition. The relaxation is reported on the console as it happens, though it is not yet recorded in the datasheet, so a design with deliberately narrow windows should be run with that output in view. The continuous selector treats its control windows the same way.

Second, every remaining condition is matched to the anchor item by item. Each dimension is standardised on the mean and sample standard deviation of the whole candidate pool rather than of a single condition, so one scaling serves every comparison. The standardised Euclidean distance from each candidate to the target item is then computed and rounded to nine decimal places, which absorbs the last-bit differences between the two language runtimes, and the nearest candidate not yet taken is assigned, with ties broken by the UTF-8 bytes of the word. A word assigned in one condition is withdrawn from every later one, so no word appears twice. Because

no random number generator is used and string comparisons are performed in byte order in both engines, the R and Python packages select an identical matched selection from identical input, a parity we verify below.

Two further matching methods can be selected when a design needs them. A covariance-aware (Mahalanobis) distance replaces the standardised Euclidean metric with one weighted by the inverse of the control dimensions' correlation matrix, so it down-weights dimensions that are correlated, such as length and orthographic neighbourhood, rather than counting their shared variance more than once (Rubin, 1980; Stuart, 2010). A globally optimal assignment, for a two-condition design, minimises the total distance over the whole pairing instead of taking the locally cheapest pair at each step (Gu & Rosenbaum, 1993; Hansen & Klopfer, 2006). Both rely on a linear-algebra routine (a matrix inverse and an assignment solver respectively) whose last bits differ between the R and Python backends, so on these two methods the engines select equivalent but not byte-identical materials; each datasheet records which case applies. The optimal method also carries a dependency the other methods do not. In R its assignment solver comes from the suggested package 'clue', installed on demand rather than alongside 'lexsync', and in Python from 'SciPy', which the package already requires.

A design may also be declared *continuous* rather than dichotomised. Instead of splitting a variable into conditions and matching, 'lexsync' selects a set that spans the variable's range evenly while holding the control dimensions near-constant, so the variable stays continuous and is analysed by regression or a mixed model rather than a between-condition contrast. This is the recommended alternative to matched dichotomies, which lose power and admit selection artefacts (Kuperman, 2015; Liben-Nowell et al., 2019). For a continuous design the datasheet reports the realised span, the predictor-control correlations and a suggested regression model.

The per-anchor procedure fixes one condition and matches the others to it, which is ideal when the manipulated dimension is statistically independent of the controls. When the manipulation is intrinsically correlated with a control dimension (as orthographic neighbourhood density is with word length and frequency), fixing one condition first can leave a residual difference on the control, because the anchor items need not have close counterparts in the other condition. For such designs 'lexsync' offers a `joint` matching method, selected with a single line in the design configuration. Rather than privileging one condition, it scores cross-condition pairs on the standardised control dimensions and greedily retains the n cheapest disjoint pairs, so only items that have a close counterpart in the opposite condition are kept. To keep the pair matrix tractable, each condition's candidate set is first reduced, in turn, to the 1,200 items closest to the other condition's centroid in standardised space. In both neighbourhood demonstrations below at least one condition's candidate set is larger than that, so the reduction is active in them. The two conditions are thereby equated on the controls even when the manipulation is confounded with them, at the cost of drawing from a slightly larger candidate pool. The method is as deterministic and cross-engine identical as the per-anchor one (costs are rounded before the byte-order tie-break), and it is the method used for the neighbourhood-density demonstrations below.

2.4 Counterbalancing and reporting

Matched items are assigned to presentation lists and a reproducible trial order, and a participant table crosses any counterbalancing factors. The trial order within each list comes from a seeded, keyed-hash shuffle: each trial is ranked by the SHA-256 digest of the string `seed|replicate|list|set|condition`, a tuple that identifies it uniquely under either counterbalancing recipe. Distinct inputs to SHA-256 behave as independent uniform draws, so ranking by the digest realises a seeded random permutation, yet the permutation is a pure

function of the design. A different seed gives a different order, and the same seed gives the same order, byte for byte, from both engines on any platform. Nothing in the ordering consults the machine, since the digest and the byte-order sort that ranks trials by it are fixed by the specification rather than by the local toolchain. That is checked rather than assumed. The parity suite runs on Linux, macOS and Windows, and on every push and pull request continuous integration generates the experiments with each engine in turn and fails if any file differs. No random-number generator is involved anywhere in the package, so there is no generator state to save or restore. The permutation is uniform, which makes it a legitimate randomisation and not merely a reproducible one. Over a range of seeds each item’s mean serial position sits at the centre of its list, and each item reaches every position about equally often. The test suite checks the centring and that every item reaches first place at some seed. A design may also request *resampling*: several disjoint matched item sets are drawn, no item reused across them, so a study can run a different item sample per participant group or demonstrate an effect across samples rather than resting on a single fixed set of items. Generalising beyond the particular items shown is a long-standing concern (Clark, 1973; Yarkoni, 2022). ‘lexsync’ then writes a match-quality report and an automated, comprehensive run log recording each stage, its parameters, the seed, package and corpus provenance and a fingerprint of every file written.

The report is deliberately built around effect sizes rather than significance tests. For each controlled dimension it gives the per-condition descriptive statistics and the standardised mean difference (Cohen’s d , the anchor condition minus the comparison condition over their pooled standard deviation), together with a 90% confidence interval on that difference. The interval is the ordinary t interval on the raw mean difference, re-expressed in pooled standard deviations. Its half-width is the .95 quantile of t on $n_1 + n_2 - 2$ degrees of freedom, times the square root of $1/n_1 + 1/n_2$, so it carries the sampling uncertainty of the two means but not that of the pooled standard deviation, and it is therefore not the non-central t interval sometimes used for d . This ordering is a considered response to a common error in nuisance control: treating a non-significant difference test as proof that a variable is matched (Sassenhagen & Alday, 2016). A non-significant test is not evidence of equivalence, and whether a real difference reaches significance depends on the number of items, so the conclusion “matched” can be manufactured by using few items. The standardised mean difference is, by contrast, a direct measure of the realised imbalance, and its confidence interval makes the sampling uncertainty (and hence the dependence on the number of items) explicit. With few items the interval is wide, so a small point estimate cannot be over-read, and the upper limit of the interval on $|d|$ is the largest imbalance still consistent with the chosen stimuli. The report also includes a two one-sided tests (TOST) equivalence test (Lakens, 2017) as a complementary verdict against the schema’s bound of 0.5 pooled standard deviations. A 90% interval falling entirely within that bound is exactly the condition under which a TOST at the .05 level declares equivalence, so the two are coherent, and the interval is the more transparent of the pair. Together they make the realised control explicit, in the spirit of the balance checks that study-specific pipelines perform by hand, and the run log makes a run auditable.

Finally, each design is accompanied by a *materials datasheet*, a machine-readable JSON record and a human-readable Markdown summary of its provenance. The record covers the corpus or item table and its checksum, the dimensions and how they were computed, the selection or matching method and its parameters, the realised control (the effect sizes and their intervals, a variance-ratio check on the spread, and the equivalence verdicts), the counterbalancing recipe, and the seed, software versions and file checksums needed to reproduce the materials exactly. It records, in addition, the size of the candidate pool each condition was selected from, so that item selection is auditable rather than opaque, and a suggested crossed mixed-model formula that treats items as a random factor (Clark, 1973), with which it fills the pre-registration

analysis plan. The datasheet also emits an auto-filled Methods paragraph and a pre-registration template with its Materials section completed. This is a direct response to the finding that experimental materials are rarely shared in a usable form (Bochynska et al., 2023): the materials, the procedure and the record of how they were made travel together.

2.5 Hardware time-locking

Of the three presentation targets the ‘scripting’ module emits from the same matched stimuli, two drive laboratory hardware. The browser-based third is described below. In the ‘PsychoPy’ script a single routine, `show_frames`, draws every timed event and binds any onset trigger to the flip on which the stimulus first appears:

```
def show_frames(win, stim, frames, port, trigger):
    if trigger is not None:
        win.callOnFlip(port.setData, trigger)
    for f in range(frames):
        if stim is not None:
            stim.draw()
        win.flip()
        if trigger is not None and f == RESET_AFTER_FRAMES:
            win.callOnFlip(port.setData, 0)
    if trigger is not None and frames <= RESET_AFTER_FRAMES:
        port.setData(0)
```

The code is held for `triggers.reset_after_frames` frames, two by default in the schema, and the reset is itself bound to a flip. An event that does not last beyond that window clears the port directly as it ends.

A mock port prints the codes when no parallel-port driver is present, so the script runs end to end with no hardware. The ‘OpenSesame’ export is a complete, plain-text `.osexp`. It is generated programmatically, so its tab-sensitive format is always valid, and it is modelled on a working experiment reported by González Alonso et al. (2025). In it the word is drawn and shown from an inline script, and the marker is sent as soon as `show()` returns, supporting both a parallel-port and a serial backend with a test-mode fallback. Because `show()` blocks until the display refresh, that marker follows the verified flip, which is the method the platform recommends. Both laboratory targets are therefore time-locked to stimulus onset, though we recommend a photodiode check in either case. Neither the matching nor the script generation imports ‘PsychoPy’ or any serial library. Generation only writes text, and the presentation software is needed solely at run time.

2.6 Paradigm generality: the trial-event model

A study-specific script typically supports one kind of trial (a fixation cross, a single word and a response), hard-coded into the experiment. ‘lexsync’ replaces that fixed structure with a declarative one. A trial is an ordered list of *events*, each a small record naming its type (a fixation, a piece of text, a mask, a blank interval, a region-by-region passage, a response or a comprehension question), its content (a literal such as a fixation cross, or a reference to a field of the trial such as the target word or the sentence), its duration, and an optional EEG trigger that may be locked to the event’s onset. A *paradigm* is a named default sequence of such events together with the trial fields it requires; a design either selects a paradigm or

supplies its own event list. All three presentation backends iterate the same list. The ‘PsychoPy’ script interprets it at run time, the ‘OpenSesame’ generator emits one block per event, and the ‘jsPsych’ generator builds a browser timeline from it. Adding a paradigm is therefore a matter of configuration rather than backend code, and all three targets present the same events in the same order. Durations are the one place where the targets can come apart. Every event length is declared in refresh frames, and the ‘PsychoPy’ script counts frames on whatever display is in use, whereas the ‘OpenSesame’ and ‘jsPsych’ generators convert that frame count to milliseconds at an assumed 60 Hz. On a display of another refresh rate the two therefore no longer describe the same interval, and the discrepancy is proportionally largest for the briefest events, such as the prime and the mask of the masked-priming paradigm. On the ‘OpenSesame’ target the onset code is also held for 10 ms before it is reset, and that wait lengthens the event it marks by the same amount. This concerns the presentation targets only. The generated files themselves remain byte-identical between the R and Python engines, which emit the same conversion.

The ‘jsPsych’ target is a single shareable HTML file that runs in any modern browser, so anyone can reproduce the exact procedure online from the same materials. The jsPsych library itself loads from a content delivery network. Because a browser cannot drive a parallel port, each onset marker is recorded in the trial’s data rather than written to hardware, and online EEG synchronisation would route it through WebSerial or a photodiode. Stimulus text is always written as data into the conditions file the experiment reads, never interpolated into the generated code, and any value containing a control character or exceeding a length bound is rejected before writing, so a malformed item cannot corrupt a script.

Items reach a design from one of three sources. The corpus source is the matching engine described above. The *table* source loads a user-supplied set of items (prime–target pairs, or sentences segmented into regions with a comprehension question), validated against the paradigm’s required fields and counterbalanced by a Latin square that shows each item once per list in a rotated condition, so a target is never repeated within a list. The *generate* source builds pseudowords by one of two methods set in `items.generation.method`. The default, `letter_substitution`, derives a length-matched, orthographically legal non-word for each real word by changing the fewest letters such that every resulting letter bigram is attested in the corpus and the form is neither a real word nor a pseudoword already issued for the set, choosing among the legal candidates the most bigram-plausible with a byte-order tie-break. Substituted letters are drawn from a to z and the search stops after two changed positions, so the generator reports a failure rather than drifting further from its base word. The alternative, `subsyllabic`, splits each word into onset, nucleus and coda constituents and swaps up to two thirds of them, rounded up, for attested alternatives of the same role and length, taking codas first and onsets last, so that the pseudoword keeps the syllabic structure of its base. Where no legal swap is available for a word, that item falls back to letter substitution. Both are deterministic orthographic approximations of ‘Wuggy’ (Keuleers & Brysbaert, 2010) that match length exactly and return byte-identical output from the R and the Python engine.

2.7 Reproducibility and FAIR compliance

‘lexsync’ declares its dependencies in a `DESCRIPTION` file for R and in pinned requirements for Python, and it sets seeds from a single configuration file. The distribution carries a ‘CITATION.cff’, an MIT licence for the code and CC BY-SA terms for the bundled corpus derivatives, with full attribution. The R package passes `R CMD check --as-cran`; the Python package builds a wheel and a source distribution that pass `twine check`. Both ship unit test suites, including a mock-engine test that confirms the generated ‘PsychoPy’ script writes its onset trigger on the word’s first flip and a structural validator for the generated `.osexp`.

3 Worked examples

We illustrate ‘lexsync’ with fifteen demonstrations. Six are corpus-matched word contrasts, three exercise other paradigms, three put the analytic choices argued for above into practice (a continuous design, a resampled one and a second pseudoword generator) and the last three reproduce designs from the published literature (see *Reproducing published designs*, below). The first four matched contrasts cross two alphabetic languages: a high- versus low-frequency contrast and a dense- versus sparse-neighbourhood contrast, each in English and in Spanish, drawn from 30,000-word lexica. The frequency contrast matches items on length, neighbourhood density and OLD20; the neighbourhood contrast matches on length and frequency. The fifth is a high- versus low-frequency contrast in Mandarin Chinese, included to show that the pipeline generalises to a logographic writing system (see *Generalisation to a logographic script*, below). The sixth, a further English contrast, matches on an orthographic syllable estimate and a bigram-frequency measure in place of the two neighbourhood measures, exercising the richer dimensions introduced with the matching algorithm above. The next three (lexical decision, priming and self-paced reading) show the trial-event model carrying word, pseudoword, paired and sentence stimuli (see *Beyond word lists*, below). Each demonstration was produced by both the R and the Python engine. All tables and figures in this section are computed at render time from the generated output. Table 1 summarises the five frequency and neighbourhood contrasts, for which a controlled-versus-manipulated report is defined. The remaining demonstrations are described in their own sections.

Table 1: *Manipulated and Controlled Standardised Mean Differences by Demonstration.* Note. Each demonstration, its manipulated dimension and the absolute standardised mean difference (Cohen’s d) of the manipulated dimension and of the most-different controlled dimension. The manipulation is large in every case; the controlled dimensions remain small.

Demonstration	Manipulated dimension	$ d $ manipulated	largest $ d $ controlled
English: frequency contrast	Frequency (Zipf)	5.27	0.04
English: neighbourhood contrast	Neighbourhood N	3.11	0.00
Spanish: frequency contrast	Frequency (Zipf)	5.55	0.08
Spanish: neighbourhood contrast	OLD20	2.42	0.00
Chinese: frequency contrast	Frequency (Zipf)	6.00	0.11

Figure 1 plots the absolute standardised mean difference of each dimension against the anchor condition. In every demonstration the manipulated dimension stands far to the right of the 0.5-standard-deviation reference bound (dashed line), while the controlled dimensions cluster near zero.

The realised control is best read from the effect sizes and their intervals. Table 2 lists, for the controlled dimensions of each demonstration, Cohen’s d against the anchor, its 90% confidence interval and the complementary TOST verdict at the 0.5- SD bound.

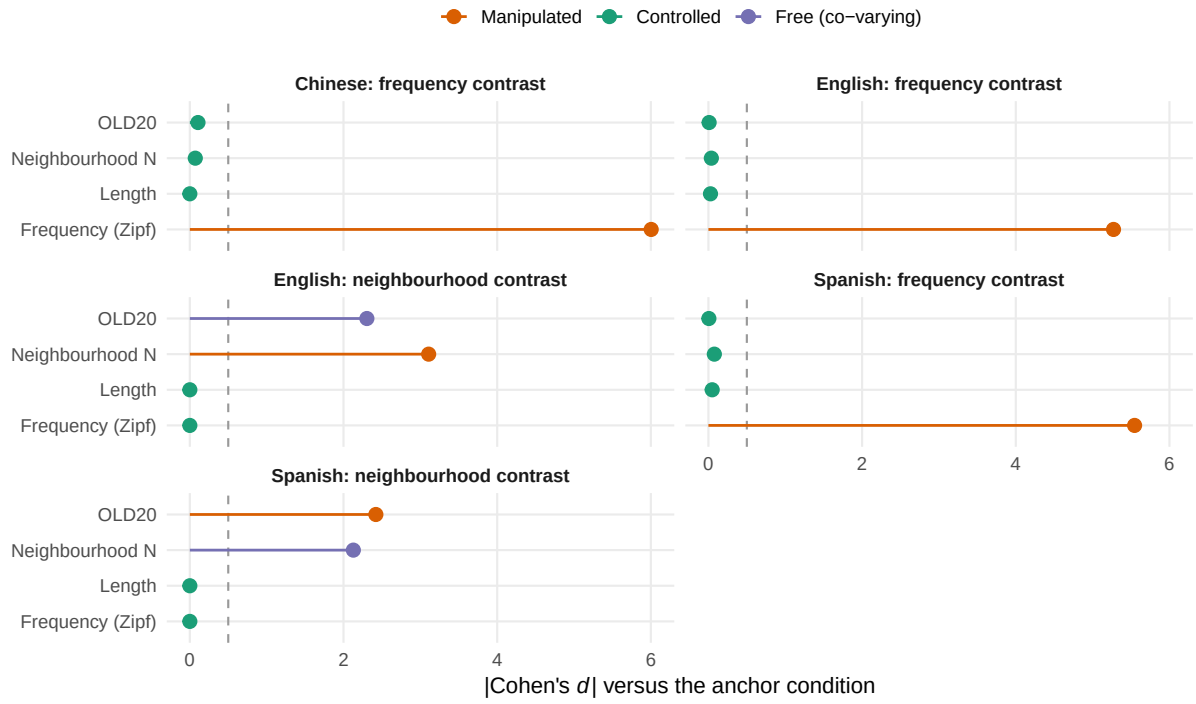


Figure 1: *Absolute Standardised Mean Differences by Dimension and Demonstration.* Note. Absolute standardised mean difference (Cohen's d) of each dimension against the anchor condition. The dashed line is the $0.5\text{-}SD$ reference bound (the smallest effect of interest for the equivalence test). Manipulated dimensions (orange) lie far to the right; matched dimensions (green) cluster near zero. In the neighbourhood contrasts OLD20 is free to co-vary (purple), reflecting its known correlation with Coltheart's N .

Table 2: *Realised Control on the Controlled Dimensions: Effect Sizes With Confidence Intervals and the Complementary Equivalence Test. Note.* For each demonstration’s controlled (matched) dimensions: the standardised mean difference (Cohen’s d) against the anchor, its 90% confidence interval, and the two one-sided tests (TOST) p value and verdict at the 0.5- SD bound. The 90% interval is the one coherent with a .05-level TOST. R engine.

Demonstration	Dimension	Cohen’s		TOST	
		d	90% CI	p	Equivalent
Chinese: frequency contrast	Length	0.000	[0.00, 0.00]	0.000	TRUE
Chinese: frequency contrast	Neighbourhood N	0.071	[-0.19, 0.33]	0.004	TRUE
Chinese: frequency contrast	OLD20	0.106	[-0.15, 0.37]	0.007	TRUE
English: frequency contrast	Length	0.027	[-0.23, 0.29]	0.002	TRUE
English: frequency contrast	Neighbourhood N	0.038	[-0.22, 0.30]	0.002	TRUE
English: frequency contrast	OLD20	0.009	[-0.25, 0.27]	0.001	TRUE
English: neighbourhood contrast	Length	0.000	[-0.26, 0.26]	0.001	TRUE
English: neighbourhood contrast	Frequency (Zipf)	0.000	[-0.26, 0.26]	0.001	TRUE
Spanish: frequency contrast	Length	0.049	[-0.21, 0.31]	0.002	TRUE
Spanish: frequency contrast	Neighbourhood N	0.077	[-0.18, 0.34]	0.004	TRUE
Spanish: frequency contrast	OLD20	0.006	[-0.26, 0.27]	0.001	TRUE
Spanish: neighbourhood contrast	Length	0.000	[-0.26, 0.26]	0.001	TRUE
Spanish: neighbourhood contrast	Frequency (Zipf)	0.000	[-0.26, 0.26]	0.001	TRUE

In each of these five demonstrations every controlled dimension’s 90% confidence interval lies within the 0.5- SD bound ($n = 80$ per condition), so the realised imbalance is small even at the interval’s far end, not merely non-significant. The TOST agrees, declaring each dimension equivalent at the .05 level, but the interval is the more informative statement because it would widen, and the claim would weaken, were the item count reduced. The neighbourhood contrasts are the demanding case: neighbourhood density is intrinsically correlated with word length and frequency, so a per-anchor match leaves a residual on length (Cohen’s $d = 0.25$). The **joint** matching method removes it: by selecting the best-matched dense-sparse pairs jointly it equates length and frequency almost exactly ($d = 0.00$ on both, with intervals tightly around zero) while the density contrast remains an order of magnitude larger ($d > 2$). The confound is therefore something the matching method can remove rather than an inherent limit on the design, and ‘lexsync’ reports the realised control transparently, with effect size and interval first and the equivalence test as a bound-referenced complement (Lakens, 2017; Sassenhagen & Alday, 2016).

The per-condition descriptive statistics make the control concrete. In the English frequency contrast (Table 3) frequency is separated by roughly one and a half Zipf units while length, neighbourhood density and OLD20 are almost identical across conditions. In the English neighbourhood contrast (Table 4) neighbourhood density is separated by an order of magnitude with length and frequency held constant; OLD20, which is mathematically related to N, moves with the manipulation rather than being held constant, as expected.

Table 3: *Descriptive Statistics per Condition: English Frequency Contrast. Note. R engine.*

Condition	Dimension	n	Mean	SD	Min	Median	Max
high frequency	Length	80	4.700	1.427	3.00	4.000	8.00
high frequency	Frequency (Zipf)	80	5.578	0.369	5.20	5.455	6.98
high frequency	Neighbourhood N	80	7.812	8.344	0.00	5.000	34.00
high frequency	OLD20	80	1.629	0.541	1.00	1.600	3.10
low frequency	Length	80	4.662	1.350	3.00	4.000	7.00
low frequency	Frequency (Zipf)	80	4.064	0.170	3.82	4.035	4.40
low frequency	Neighbourhood N	80	7.513	7.480	0.00	5.000	24.00
low frequency	OLD20	80	1.624	0.518	1.00	1.600	2.70

Table 4: *Descriptive Statistics per Condition: English Neighbourhood-Density Contrast. Note. R engine.*

Condition	Dimension	n	Mean	SD	Min	Median	Max
dense neighbourhood	Length	80	4.787	0.650	4.00	5.000	6.00
dense neighbourhood	Frequency (Zipf)	80	4.249	0.311	3.81	4.225	5.15
dense neighbourhood	Neighbourhood N	80	8.363	3.530	5.00	7.500	20.00
dense neighbourhood	OLD20	80	1.448	0.197	1.00	1.500	1.75
sparse neighbourhood	Length	80	4.787	0.650	4.00	5.000	6.00
sparse neighbourhood	Frequency (Zipf)	80	4.249	0.311	3.81	4.225	5.15
sparse neighbourhood	Neighbourhood N	80	0.525	0.503	0.00	1.000	1.00
sparse neighbourhood	OLD20	80	1.970	0.253	1.60	1.900	2.95

3.1 Generalisation to a logographic script

The orthographic-neighbourhood measures operate on character strings, so the pipeline is not tied to alphabetic writing. To show this concretely, the fifth demonstration is a high- versus low-frequency contrast in Mandarin Chinese. Its lexicon comes from the same corpus package as the others (`wordfreq`’s Chinese data (Speer, 2022)), and the two conditions are two-character words, the dominant word form in Chinese. Here ‘length’ is the number of characters, and Coltheart’s N and OLD20 are computed over characters rather than letters; we refer to them as character-level neighbourhood measures. Adding the demonstration required no change to the matching, counterbalancing or scripting code: a design configuration names the Chinese lexicon and a glyph-complete font, and the existing machinery does the rest.

Table 5 shows the result. Frequency is separated by about one and a half Zipf units (Cohen’s $d = 6$), while character-length is held constant at two and the character-level neighbourhood measures are equated across conditions (each with a 90% confidence interval on d lying within the $0.5\text{-}SD$ bound, and correspondingly TOST-equivalent). Chinese two-character words have dense orthographic neighbourhoods (tens of single-character substitutions each, an order of magnitude more than the alphabetic words above), and ‘lexsync’ equates this dense, skewed distribution as readily as the sparse alphabetic one. The generated ‘PsychoPy’ and ‘OpenSesame’ experiments set a CJK font, so the characters render directly. The selection is byte-identical across the R and Python engines, exactly as for the alphabetic demonstrations.

Table 5: *Descriptive Statistics per Condition: Mandarin Chinese Frequency Contrast. Note.* Length is in characters; neighbourhood N and OLD20 are character-level measures. R engine.

Condition	Dimension	n	Mean	SD	Min	Median	Max
high frequency	Length	80	2.000	0.000	2.0	2.000	2.00
high frequency	Frequency (Zipf)	80	5.324	0.294	5.0	5.245	6.56
high frequency	Neighbourhood N	80	69.825	64.807	8.0	51.500	307.00
high frequency	OLD20	80	1.038	0.111	1.0	1.000	1.55
low frequency	Length	80	2.000	0.000	2.0	2.000	2.00
low frequency	Frequency (Zipf)	80	3.810	0.201	3.5	3.810	4.20
low frequency	Neighbourhood N	80	65.662	52.159	14.0	51.500	199.00
low frequency	OLD20	80	1.028	0.074	1.0	1.000	1.25

3.2 Cross-engine parity

Because the matcher is deterministic the two engines select the same matched (word, condition) pairs rather than merely similar ones, and because the trial order is a keyed-hash function of the design they also place those pairs in the same order. Across the five matched word contrasts all 800 selected (word, condition) pairs were identical between the R and the Python engine, and every statistic they report agrees to the precision shown. That check runs when this manuscript is rendered. The test suite makes a stricter one. For each of the fifteen design configurations shipped with the package it regenerates the Python selection and requires it to agree with the committed R selection on the selected item, its condition and the set the matcher paired it into, and then on every other column the two files share, value for value. Continuous integration goes further and runs each engine into its own directory, comparing the two trees of generated experiments recursively, so all 75 files (for every design, a ‘PsychoPy’ script and its conditions table, an ‘OpenSesame’ experiment and its loop table, and a ‘jsPsych’ page) must agree byte for byte. The same job regenerates the R selections first and fails if they have drifted from the committed ones, so the comparison cannot pass against a stale reference. The covariance-aware and optimal matching methods remain the exceptions, for the reason given above. Table 6 makes this concrete for the English frequency contrast: the standardised mean difference of each dimension, computed independently in the two languages, coincides to the precision shown.

Table 6: *Cross-Engine Parity for the English Frequency Contrast. Note.* Each dimension’s standardised mean difference (Cohen’s d) against the anchor, computed independently by the R and Python engines. The columns are identical to three decimal places.

Dimension	Cohen’s d (R engine)	Cohen’s d (Python engine)
Length	0.027	0.027
Frequency (Zipf)	5.272	5.272
Neighbourhood N	0.038	0.038
OLD20	0.009	0.009

3.3 Beyond word lists: three paradigms

The same engine builds tasks that are not word lists. Three further demonstrations, each in English, exercise a different stimulus structure and trial procedure through the trial-event model.

A lexical-decision design contrasts 60 real words with 60 pseudowords generated to match them in length; the generated non-words are orthographically legal and absent from the corpus, and length is equated by construction (Cohen’s $d = 0.00$). A priming design draws 12 prime–target pairs from an item table and contrasts related with unrelated primes, its trial inserting a brief prime and a mask before the onset-locked target. A self-paced reading design presents 10 sentences region by region, the grammatical–ungrammatical manipulation falling on the onset-locked critical region and followed by a comprehension question. Each design compiles from the same event list to a runnable ‘PsychoPy’ script, a valid ‘OpenSesame’ experiment and a browser-based ‘jsPsych’ experiment, and the R and Python engines select identical materials in every case, including, for lexical decision, the very same deterministically generated pseudowords.

3.4 Selection modes for the intended analysis

Three further demonstrations put the analytic choices argued for above into practice, all drawn from the same English lexicon as the frequency contrast. The continuous design abandons the high-low split. Its 80 items span 3.05 Zipf units of frequency evenly while length, neighbourhood density and OLD20 are held near-constant, and the largest correlation between the predictor and any control dimension is $|r| = 0.16$. The datasheet accordingly proposes a mixed model with frequency entered continuously rather than a between-condition contrast. The resampled design draws the same frequency contrast into 3 disjoint matched sets, 240 items in all, with no word reused across sets, so a study can give a different item sample to each participant group and treat items as the random factor they are (Clark, 1973; Yarkoni, 2022). Control on length, neighbourhood density and OLD20 holds across the pooled sets, the largest absolute standardised difference on a controlled dimension being 0.04. The third repeats the lexical-decision design with the `subsyllabic` generator in place of the default, over the same set of 60 base words, so the two pseudoword methods can be compared like for like. They have no pseudoword in common. For all three the R and Python engines select identical materials, and each datasheet records the selection as byte-identical across the engines.

3.5 Reproducing published designs

The demonstrations so far are illustrative. To show that ‘lexsync’ reproduces designs from the published literature, we recreated three studies that span the workflow’s range: a longitudinal EEG study of artificial language learning, a classic factorial lexical-decision design and a masked-priming design. Each was rebuilt to the original’s matching *criteria* (its manipulated and controlled dimensions and their tolerances) from a contemporary, redistributable corpus, rather than by copying the authors’ word lists. Table 7 summarises the three, and the two engines reproduce each one identically.

Table 7: *Three Published Designs Reproduced to Criteria. Note.* For each reproduction: the structure rebuilt, the number of generated items, and the largest standardised mean difference (Cohen’s d) remaining on a controlled (matched) dimension, with in brackets the upper limit of its 90% confidence interval, that is the largest imbalance still consistent with the selected items. The manipulations are large by construction; for Andrews these are frequency and neighbourhood size, reported in the text. The Rastle design is a priming layout with no corpus-matched control dimension.

Reproduction	Design rebuilt	Items	Largest controlled $ d $ (upper 90% limit)
González Alonso et al. (2025)	Gender assignment: feminine vs masculine, matched on frequency and length	96	0.02 (0.36)
Andrews (1989)	Frequency x neighbourhood size: 2x2 factorial, cells matched on length	72	0.21 (0.77)
Rastle et al. (2004)	Masked priming: transparent / opaque / orthographic, vs unrelated baseline	24	—

González Alonso et al. (2025) administered a natural-language gender-assignment task in which corpus nouns, half feminine and half masculine, were matched to the study’s nouns on word frequency (within mean $\pm SD/9$) and number of letters (within mean $\pm 2 SD$). ‘lexsync’ reproduces this design directly: the same per-dimension tolerance windows are declared in the design file and applied by the matcher, and feminine and masculine words are equated on frequency and length to a largest controlled Cohen’s d of 0.02 across 96 items. Because the bundled Spanish corpus carries no lexical-gender field, gender is approximated by the canonical ending (*-a*, *-o*) rather than looked up as the original did in EsPal (Duchon et al., 2013); the demonstration is therefore of the matched selection and its realised control, the part ‘lexsync’ automates, not of the authors’ exact items.

Andrews (1989) crossed word frequency with orthographic neighbourhood size in a 2×2 design, run in lexical decision and in naming, and found that a large neighbourhood facilitates low-frequency words disproportionately. ‘lexsync’ rebuilds the factorial structure over short words, equating all four cells on length (largest controlled $|d| = 0.21$, upper 90% limit 0.77) while separating the manipulated dimensions widely. With 18 items per cell the interval is wide, and length is not shown equivalent at the 0.5- SD bound (TOST $p = 0.192$). The realised imbalance is small as a point estimate, but at that cell size the upper limit is what the data support, and the datasheet records it rather than letting the point estimate stand alone. Frequency d reaches 5.2 and neighbourhood d reaches 3.3. Frequencies come from a contemporary corpus rather than the Kučera–Francis counts of the original, so the realised bands differ while the factorial design does not.

Rastle et al. (2004) used masked priming to dissociate three kinds of prime–target relationship, transparent morphological (*cleaner*–CLEAN), opaque morphological (*corner*–CORN) and purely orthographic (*brothel*–BROTH), each against an unrelated baseline, arguing for an early morpho-orthographic segmentation stage blind to meaning. ‘lexsync’ expresses the design with its priming paradigm: 12 targets are each shown with a related prime of their assigned type and an unrelated, length-matched control, Latin-square counterbalanced across 2 lists so a target appears once per list, with the prime type carried as a between-items factor for analysis. This is a representative item set following the prime-type structure of the original, not its full materials.

4 Discussion

‘lexsync’ unifies steps that are usually performed in separate tools and separate languages. Its principal contribution is integration. Multidimensional matching exists in ‘LexOPS’ and ‘LIBRA’, and precise presentation exists in ‘PsychoPy’ and ‘OpenSesame’, but the path from a multilingual corpus to a hardware-timed, cross-platform experiment has not before been available as a single installable, dual-language tool. Two features are worth emphasising. First, the deterministic matcher selects identical matched (word, condition) sets in the R and Python engines, with the reported match statistics agreeing to the precision shown, and the keyed-hash trial order makes every generated experiment file byte-identical between them; the sole exceptions remain the covariance-aware (Mahalanobis) and optimal matching methods, whose linear-algebra backends differ in their last bits. A laboratory can therefore adopt ‘lexsync’ in whichever ecosystem it prefers without fear of divergent stimulus selection. Second, both laboratory targets place every onset marker at the stimulus flip (flip-locked in ‘PsychoPy’, and written immediately after the refresh-blocking `show()` in ‘OpenSesame’), rather than sending a trailing block of markers in which only the first is even approximately aligned, a concrete methodological improvement for ERP research.

Some limitations remain. The Chinese demonstration shows that the pipeline generalises to a logographic script (corpus access, dimension derivation, matching, counterbalancing and CJK-font script generation all work unchanged), but the neighbourhood measures it equates there (Coltheart’s N and OLD20) are computed at the character level. They do not capture the sub-character structure that also shapes Chinese reading, such as radical, stroke-count and phonetic-radical information. Such measures are not intrinsic to the method; they enter as additional script-specific dimensions through the same extensible dimension system, and supplying them is left to future work. A photodiode check remains advisable for any onset-critical study, as for any software trigger. The example item tables for the paired and sentence paradigms are deliberately small, intended to exercise and document the pipeline rather than to constitute validated materials; researchers supply their own.

Several further extensions follow directly from the motivation and slot into the existing module structure. The web target above could be joined by a ‘PCIbex’ reading study and by client-side hardware synchronisation (WebSerial or a photodiode patch), so that online studies inherit the same onset markers the laboratory targets write. The dimension set already extends beyond orthography to an orthographic syllable estimate and a corpus-derived bigram-frequency (phonotactic-probability) measure, and it could grow further to predictability dimensions such as n-gram or language-model surprisal. Semantic dimensions such as concreteness, age of acquisition and valence can be joined on through the norm-table connector. The norm tables themselves have to be obtained separately, because their licences vary. Sampling matched item sets per participant, rather than per study, would extend the resampling already provided.

5 Extending lexsync

In keeping with the design, extensions have well-defined insertion points. To add a paradigm, add an entry to the registry in `paradigms` giving its default event sequence, the trial fields it requires and its counterbalancing recipe, and all three presentation backends then render it with no further code. To add a lexical dimension, compute it in `querying` (alongside `add_neighbourhood`) and list it in the dimension table in `config/schema.yaml`, in both packages. To add a presentation target, add an `export_<target>()` function in `scripting` that walks the same rendered event list. To add a corpus or language, add an entry to

`corpora/registry.yaml`, and no code change is required for SUBTLEX-family or ‘wordfreq’ sources.

6 Availability and open practices

‘lexsync’ is available as an R package and a Python package, with a shared corpus registry and the reproducible source of this manuscript. A browser application for each engine (a Shiny app for the R package and a Streamlit app for the Python one) lets a researcher assemble a design, run the verified pipeline and export the design configuration together with the R, Python and command-line code that reproduces it, so the interface itself yields shareable, reproducible artefacts. The code is released under the MIT licence and the bundled corpus derivatives under CC BY-SA 4.0, with full attribution and retrieval dates recorded alongside the data. The repository is cited from the package metadata of both engines, from the citation file and from the machine-readable software description, so a reader arrives at the same source whichever record they follow.

7 References

- Andrews, S. (1989). Frequency and neighborhood effects on lexical access: Activation or search? *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 15(5), 802–814. <https://doi.org/10.1037/0278-7393.15.5.802>
- Bochynska, A., Keeble, L., Halfacre, C., Casillas, J. V., Champagne, I.-A., Chen, K., Röthlisberger, M., Buchanan, E. M., & Roettger, T. B. (2023). Reproducible research practices and transparency across linguistics. *Glossa Psycholinguistics*, 2(1). <https://doi.org/10.5070/G6011239>
- Brysbaert, M., & New, B. (2009). Moving beyond Kučera and Francis: A critical evaluation of current word frequency norms and the introduction of a new and improved word frequency measure for American English. *Behavior Research Methods*, 41(4), 977–990. <https://doi.org/10.3758/BRM.41.4.977>
- Cai, Q., & Brysbaert, M. (2010). SUBTLEX-CH: Chinese word and character frequencies based on film subtitles. *PLoS ONE*, 5(6), e10729. <https://doi.org/10.1371/journal.pone.0010729>
- Clark, H. H. (1973). The language-as-fixed-effect fallacy: A critique of language statistics in psychological research. *Journal of Verbal Learning and Verbal Behavior*, 12(4), 335–359. [https://doi.org/10.1016/S0022-5371\(73\)80014-3](https://doi.org/10.1016/S0022-5371(73)80014-3)
- Coltheart, M., Davelaar, E., Jonasson, J. T., & Besner, D. (1977). Access to the internal lexicon. In S. Dornic (Ed.), *Attention and performance VI* (pp. 535–555). Erlbaum.
- Coretta, S., Casillas, J. V., Roessig, S., Franke, M., Ahn, B., Al-Hoorie, A. H., Al-Tamimi, J., Alotaibi, N. E., AlShakhori, M. K., Altmiller, R. M., Arantes, P., Athanasopoulou, A., Baese-Berk, M. M., Bailey, G., Sangma, C. B. A., Beier, E. J., Benavides, G. M., Benker, N., BensonMeyer, E. P., ... Roettger, T. B. (2023). Multidimensional signals and analytic flexibility: Estimating degrees of freedom in human-speech analyses. *Advances in Methods and Practices in Psychological Science*, 6(3). <https://doi.org/10.1177/25152459231162567>
- Cuetos, F., González-Nosti, M., Barbón, A., & Brysbaert, M. (2011). SUBTLEX-ESP: Spanish word frequencies based on film subtitles. *Psicológica*, 32(2), 133–143.
- Duchon, A., Perea, M., Sebastián-Gallés, N., Martí, A., & Carreiras, M. (2013). EsPal: One-stop shopping for Spanish word properties. *Behavior Research Methods*, 45(4), 1246–1258. <https://doi.org/10.3758/s13428-013-0326-1>
- González Alonso, J., Bernabeu, P., Silva, G., DeLuca, V., Poch, C., Ivanova, I., & Rothman, J. (2025). Starting from the very beginning: Unraveling third language (L3) development with longitudinal data from artificial language learning and EEG. *International Journal of Multilingualism*, 22(1), 119–142. <https://doi.org/10.1080/14790718.2024.2415993>
- Gu, X. S., & Rosenbaum, P. R. (1993). Comparison of multivariate matching methods: Structures, distances, and algorithms. *Journal of Computational and Graphical Statistics*, 2(4), 405–420. <https://doi.org/10.1080/10618600.1993.10474623>
- Hansen, B. B., & Klopfer, S. O. (2006). Optimal full matching and related designs via network flows. *Journal of Computational and Graphical Statistics*, 15(3), 609–627. <https://doi.org/10.1198/106186006X137047>
- Heuven, W. J. B. van, Mandera, P., Keuleers, E., & Brysbaert, M. (2014). SUBTLEX-UK: A new and improved word frequency database for British English. *Quarterly Journal of Experimental Psychology*, 67(6), 1176–1190. <https://doi.org/10.1080/17470218.2013.850521>
- Itkonen, S., Häikiö, T., Vainio, S., & Lehtonen, M. (2024). LASTU: A psycholinguistic search tool for Finnish lexical stimuli. *Behavior Research Methods*, 56(6), 6165–6178. <https://doi.org/10.3758/s13428-024-02347-x>
- Keuleers, E., & Brysbaert, M. (2010). Wuggy: A multilingual pseudoword generator. *Behavior Research Methods*, 42(3), 627–633. <https://doi.org/10.3758/BRM.42.3.627>
- Kuperman, V. (2015). Virtual experiments in megastudies: A case study of language and emotion. *Quarterly Journal of Experimental Psychology*, 68(8), 1693–1710. <https://doi.org/>

10.1080/17470218.2014.989865

- Lakens, D. (2017). Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4), 355–362. <https://doi.org/10.1177/1948550617697177>
- Liben-Nowell, D., Strand, J., Sharp, A., Wexler, T., & Woods, K. (2019). The danger of testing by selecting controlled subsets, with applications to spoken-word recognition. *Journal of Cognition*, 2(1), 2. <https://doi.org/10.5334/joc.51>
- Lintz, E. N., Lim, P. C., & Johnson, M. R. (2021). A new tool for equating lexical stimuli across experimental conditions. *MethodsX*, 8, 101545. <https://doi.org/10.1016/j.mex.2021.101545>
- Mathôt, S., Schreij, D., & Theeuwes, J. (2012). OpenSesame: An open-source, graphical experiment builder for the social sciences. *Behavior Research Methods*, 44(2), 314–324. <https://doi.org/10.3758/s13428-011-0168-7>
- Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, 51(1), 195–203. <https://doi.org/10.3758/s13428-018-01193-y>
- Rastle, K., Davis, M. H., & New, B. (2004). The broth in my brother’s brothel: Morpho-orthographic segmentation in visual word recognition. *Psychonomic Bulletin & Review*, 11(6), 1090–1098. <https://doi.org/10.3758/BF03196742>
- Roettger, T. B. (2019). Researcher degrees of freedom in phonetic research. *Laboratory Phonology*, 10(1), Article 1. <https://doi.org/10.5334/labphon.147>
- Roettger, T. B., Winter, B., & Baayen, H. (2019). Emergent data analysis in phonetic sciences: Towards pluralism and reproducibility. *Journal of Phonetics*, 73, 1–7. <https://doi.org/10.1016/j.wocn.2018.12.001>
- Rubin, D. B. (1980). Bias reduction using Mahalanobis-metric matching. *Biometrics*, 36(2), 293–298. <https://doi.org/10.2307/2529981>
- Sassenhagen, J., & Alday, P. M. (2016). A common misapplication of statistical inference: Nuisance control with null-hypothesis significance tests. *Brain and Language*, 162, 42–45. <https://doi.org/10.1016/j.bandl.2016.08.001>
- Speer, R. (2022). *rspeer/wordfreq: v3.0* (Version 3.0.2) [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.7199437>
- Stuart, E. A. (2010). Matching methods for causal inference: A review and a look forward. *Statistical Science*, 25(1), 1–21. <https://doi.org/10.1214/09-STS313>
- Taylor, J. E., Beith, A., & Sereno, S. C. (2020). LexOPS: An R package and user interface for the controlled generation of word stimuli. *Behavior Research Methods*, 52(6), 2372–2382. <https://doi.org/10.3758/s13428-020-01389-1>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, IJ. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., ... Mons, B. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Yarkoni, T. (2022). The generalizability crisis. *Behavioral and Brain Sciences*, 45, e1. <https://doi.org/10.1017/S0140525X20001685>
- Yarkoni, T., Balota, D., & Yap, M. (2008). Moving beyond Coltheart’s N: A new measure of orthographic similarity. *Psychonomic Bulletin & Review*, 15(5), 971–979. <https://doi.org/10.3758/pbr.15.5.971>